Evolving CIFAR-10 CNNs with CoDeepNEAT and Compressing Convolutions with V:N:M

Systems Engineering for Deep Learning
Course Project Report

Abstract

This project evolves a convolutional network for CIFAR-10 with CoDeepNEAT, fully trains the winner, then stores the convolution weights in the VENOM **V:N:M** format. NVIDIA Sparse Tensor Cores only accept a **2:4** pattern. VENOM keeps a sparser **2:**M pattern in software and still presents a 2:4 tile to the hardware. The official CUDA library for that last step is **Spatha**. We did not link Spatha. Instead we implemented the prune, pack and unpack path in Python / PyTorch and ran inference on a reconstructed dense `nn.Conv2d`. That is a correct accuracy and storage study; it is not a Sparse Tensor Core speedup.

A short Colab search produced blueprint 608 at **83.16%**. A stronger 80-generation search on an RTX 6000 Ada produced blueprint **1485**. After 150 epochs at feature multiplier 3 the saved checkpoint is **87.38%** (peak during training **88.18%**). Magnitude V:N:M with V=8 and eight masked recover epochs then gives the results in Table 1. **2:4 recovers to dense accuracy.** 2:8 is a usable size/accuracy trade-off. 2:16 does not recover under one-shot magnitude pruning.

Table 1: V:N:M results on the Ada winner (V=8, N=2).

Pattern	Sparsity	Zero-shot	After recover	Packed vs dense convs
Dense	0%	87.38%	—	44.4 MB
2:4	50%	42.51%	**88.29%**	**3.16**× (14.0 MB)
2:8	75%	16.86%	**81.72%**	**6.32**× (7.0 MB)
2:16	87.5%	11.39%	23.65%	12.63× (3.5 MB)

Contents

1 Introduction **3**

2 Background **3**

2.1 CoDeepNEAT 3

2.2 From unstructured prune to N:M to V:N:M 3

2.3 What Spatha would have done 4

3 Method **4**

3.1 Pipeline 4
3.2 Evolution configs 5
3.3 Winner architecture (blueprint 1485) 6

4 Custom Python path in place of Spatha 8
4.1 Why a conv must become a matrix 9
4.2 Magnitude V:N:M prune 9
4.3 Pack and unpack — the format Spatha would consume 10
4.4 Putting it back into Conv2d — the Spatha substitute 11

5 Experimental setup 11

6 Results 12
6.1 Search versus full-train 12
6.2 V:N:M on the Ada winner 14
6.3 Colab pilot versus Ada 16
6.4 What we are not claiming 16

7 Discussion 17

8 Conclusions 17

9 Reproducibility 17

A Code map 18

1 Introduction

The project has two parts that are usually studied separately.

1. **Architecture search.** CoDeepNEAT evolves small reusable *modules* and *blueprints* that wire those modules into a full CNN. Fitness is short-train accuracy on CIFAR-10. The winner is then trained for many more epochs, optionally with a larger channel count (feature multiplier).
2. **Hardware-aware sparsity.** After the network exists, we want fewer stored weights, in a pattern Sparse Tensor Cores can execute. VENOM's V:N:M format is that pattern. Spatha is the CUDA kernel that multiplies packed V:N:M by a dense activation matrix.

We combined them on a real evolved CNN, not a hand-designed ResNet. The missing public piece is a V:N:M convolution kernel: VENOM's paper and Spatha cover linear-layer SpMM, not `Conv2d`. The engineering question was therefore:

> How do we put an evolved convolution into V:N:M, measure accuracy and packed size, and still run the model, without building a CUDA kernel?

The answer is a Python pack/unpack path that writes a sparse-but-dense tensor back into `nn.Conv2d.weight`. Section 4 explains that code in detail.

2 Background

2.1 CoDeepNEAT

CoDeepNEAT (*Evolving Deep Neural Networks*, Miikkulainen et al., arXiv:1703.00548) keeps two populations:

- **Modules** are small DAGs of layers (convolution, activation, batch-norm, dropout, pooling). Similar modules are grouped into species.
- **Blueprints** are DAGs whose nodes point at a *module species*, not a fixed module. At evaluation time each blueprint node samples a module from that species.

Fitness is the accuracy of a short training run. We used the PyTorch implementation sash-a/CoDeepNEAT. Search code was not rewritten; we added configs, a V:N:M package, and the evaluation script.

A **feature multiplier** (*fm*) rebuilds the same genotype with every channel count scaled. Search runs at $fm = 1$. Full-train on Ada used $fm \in \{1, 3\}$. Blueprint 1485 at $fm = 1$ has channels 190 / 360 / 720; at $fm = 3$ the same graph is 329 / 624 / 1248.

2.2 From unstructured prune to $N{:}M$ to V:N:M

Unstructured pruning can zero any weight. Accuracy holds, but GPUs cannot skip scattered zeros. Structured pruning removes whole channels or tiles and is hardware-friendly, but hurts accuracy more.

N:M **sparsity** is the compromise used by NVIDIA Sparse Tensor Cores: in every group of M consecutive weights, keep N. Current hardware implements **2:4** only (50% sparse). A 2:8 or 2:16 network is 75% or 87.5% sparse, but the GPU cannot run those patterns natively.

VENOM (Castro et al., *VENOM: A Vectorized N:M Format*, arXiv:2310.02065) cuts a weight matrix into slabs of V rows by M columns. Inside each slab:

1. Keep the **4 columns** with the largest column-wise magnitude. Their original positions are `col_loc`.
2. On the remaining $V \times 4$ tile, keep **2** entries per row ($N = 2$). Their positions among the 4 kept columns are `m_idx`.
3. Store the 2 nonzero values per row as `values`.

The hardware still sees 2:4. Extra sparsity lives in the dropped columns. Storage is:

- `values` — fp16, shape $(R_{\text{pad}}, n_{\text{blocks}}, 2)$
- `m_idx` — 2 bits per entry
- `col_loc` — one byte per kept column when $M \leq 256$

2.3 What Spatha would have done

Spatha is VENOM's CUDA SpMM: packed V:N:M weights × dense activations, on Ampere and newer Sparse Tensor Cores. Linking it would require:

- building the Spatha extension against the server CUDA toolkit;
- replacing each `nn.Conv2d` GEMM with an unfold (im2col) plus `spatha.spmm`;
- padding C_{out} to a multiple of V and $C_{\text{in}} \cdot k_h \cdot k_w$ to a multiple of M;
- accepting that CIFAR feature maps are small, so kernel launch overhead can erase the theoretical speedup.

We decided **not** to link Spatha. The Python path below is the substitute: same prune rule, same packed arrays, dense `Conv2d` for the actual multiply.

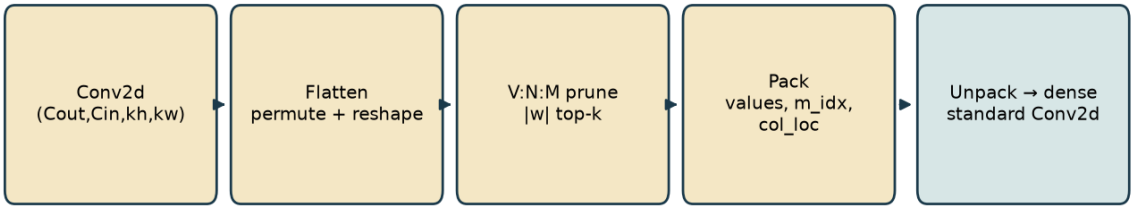
3 Method

3.1 Pipeline

```
CIFAR-10
    |
    v
CoDeepNEAT evolution (short train = fitness)
    |
    v
```

```
Select top blueprints  ->  ft.py  (150 epochs, fm = 1 and 3)
    |
    v
Winner checkpoint  bp-1485_fm-3-best-3.model
    |
    v
vnm/prune_eval.py
    flatten each Conv2d -> V:N:M prune -> pack -> unpack
    write pruned weights back into nn.Conv2d
    optional 8-epoch Adam recover with the mask reapplied
    report accuracy and packed bytes
```

Python stand-in for Spatha: pack V:N:M, then run dense Conv2d



Spatha would replace the last box with a CUDA SpMM on packed values. We keep PyTorch Conv2d.

Figure 1: Spatha would replace the last box with a CUDA SpMM on the packed arrays. We unpack to a dense tensor and call standard `Conv2d`.

3.2 Evolution configs

Table 2: Search configurations.

	Colab baseline	Ada strong search
Hardware	T4 (Google Colab)	RTX 6000 Ada, IIT lab (`AMD6KADA`)
Generations	12	80
Blueprint / module pop	20 / 50	25 / 80
Evals per blueprint	1×2 epochs	4×12 epochs
Batch	128	256
Channel range	32–256	64–512
Module species	4	6
Full-train	1 net, fm=1, 60 epochs	3 nets, $fm \in \{1, 3\}$, 150 epochs
Evolved data augmentation	off	off
Parameter cap	50 M	50 M (oversized nets dropped)

Search used Adam, no evolved DA, and CIFAR-10 as provided by torchvision. During evolution the train set is split so that "test" is a validation slice; full-train uses the official 50k / 10k split.

3.3 Winner architecture (blueprint 1485)

Every blueprint node samples **module 1463** from **species 19**. The module is two 3×3 convolutions: a narrowing conv with batch-norm, dropout 0.1 and LeakyReLU, then a widening conv with 3×3 max-pool (stride 3), dropout ≈ 0.37 and ReLU6. The blueprint DAG is:

- node **0** (stem) $\rightarrow$ skip and side branch;
- node **2** (side branch, same module);
- **concat** at `1485_agg` (lossy merge is off; channels stack);
- node **1** (head module);
- flatten + linear to 10 classes.

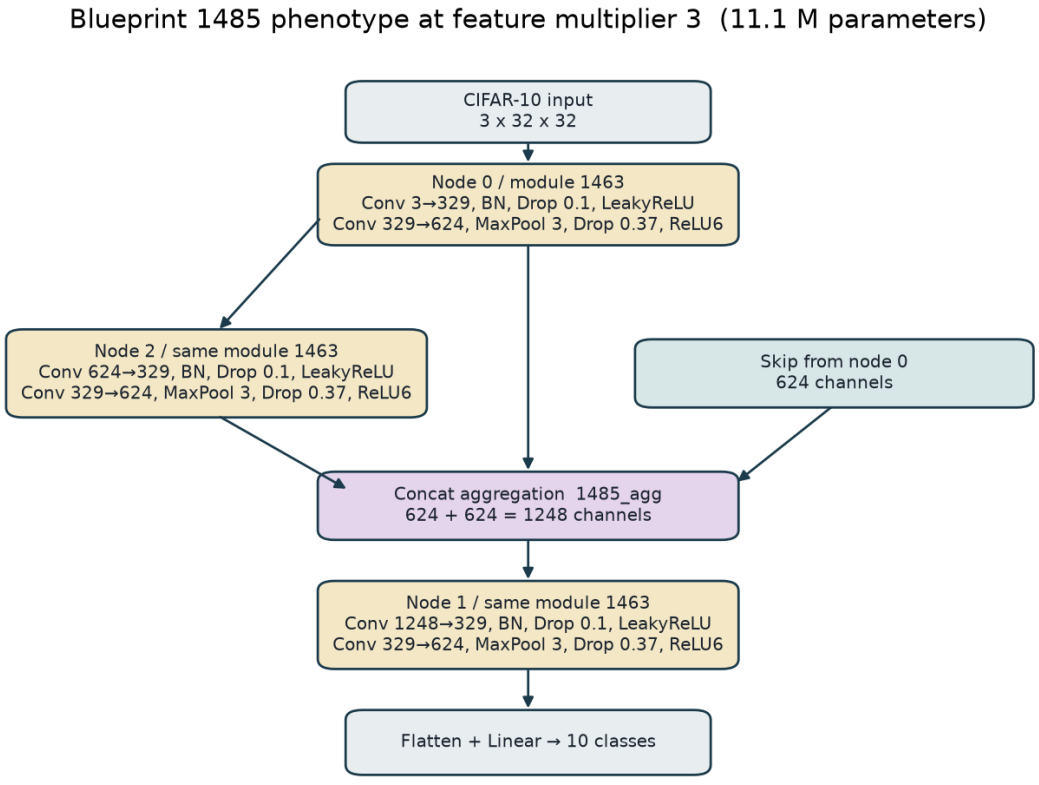


Figure 2: Phenotype at feature multiplier 3, the 11.1 M-parameter network that was pruned. Concatenation after the skip produces the 1248-channel layer.

The Graphviz dump from the Ada run (channels shown at $fm = 1$) is the official architecture figure.

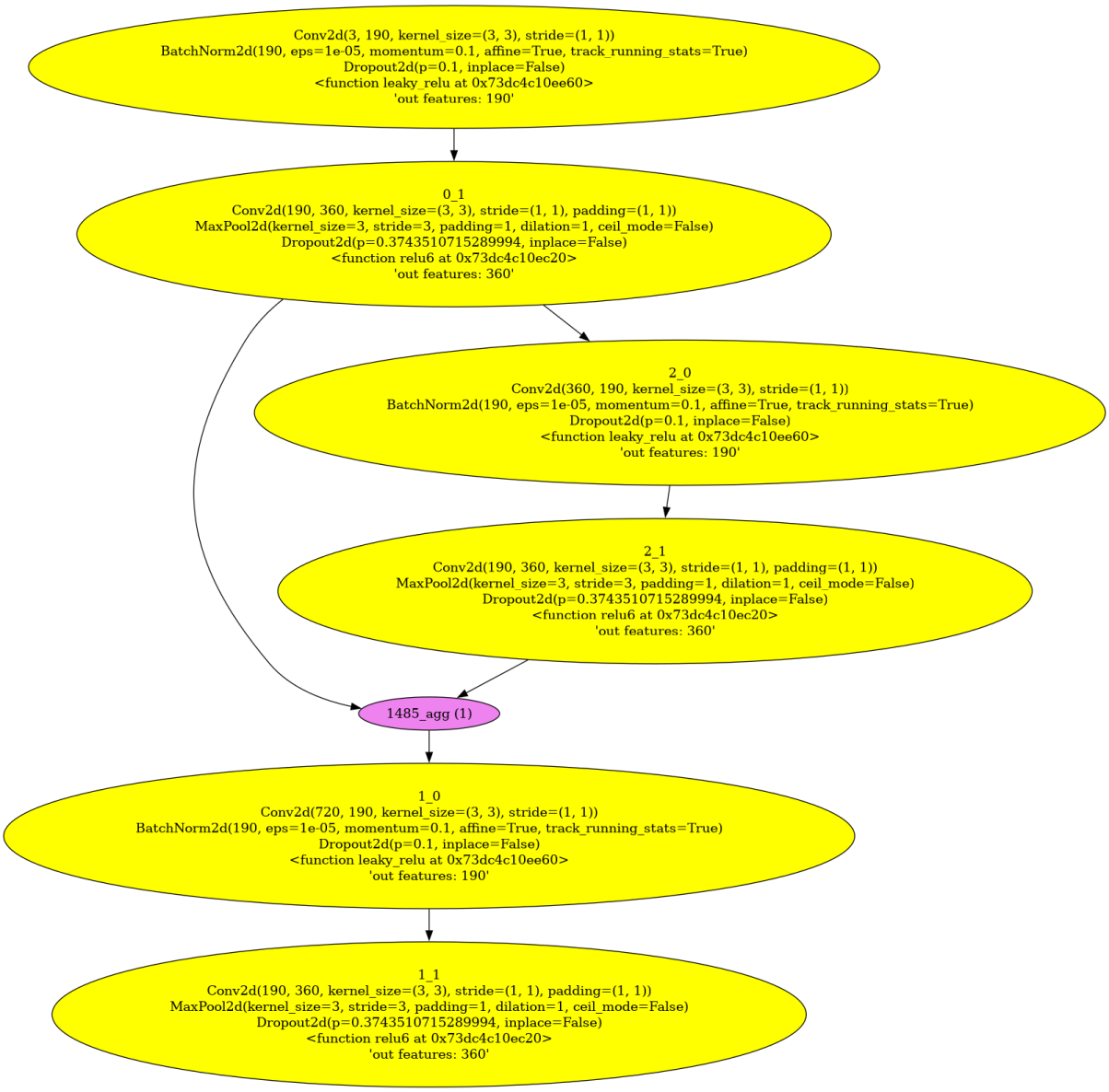


Figure 3: CoDeepNEAT phenotype visualisation, `top3_g15_blueprint_i1485_phenotype`. Yellow nodes are module layers. The violet node is the concat aggregation.

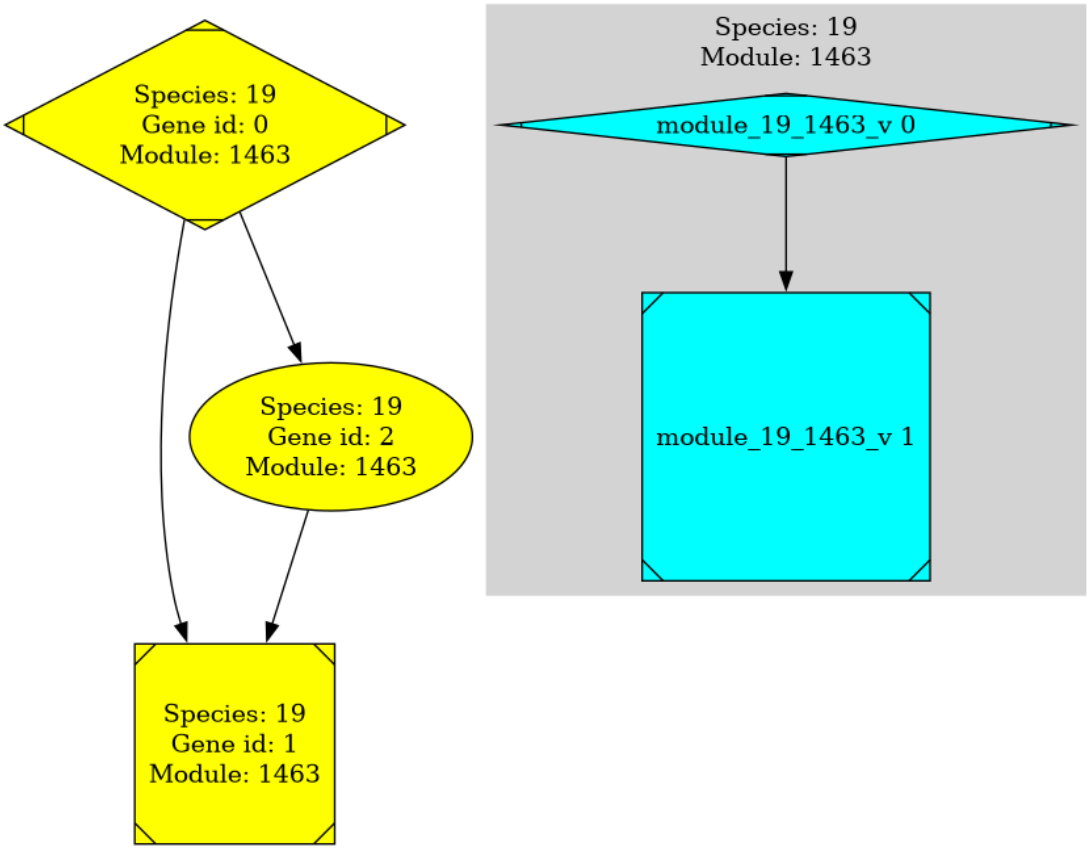


Figure 4: Genotype: all three blueprint nodes point at species 19 / module 1463. The grey inset is that module's two-vertex DAG.

Genotype versus phenotype: the genome only says "use species 19, skip from node 0 to 1, concat." The phenotype is the concrete `nn.Module` after a module is sampled and channels are multiplied by *fm*. Full-train does not change the wiring; it only scales width.

At $fm = 3$ the six convolutions are listed in Table 3. The 329×1248 layer dominates storage. Biases, BatchNorm and the final linear stay dense.

Table 3: Convolution layers of blueprint 1485 at $fm = 3$.

Layer (eval name)	Weight shape	Role
`model.sequential.0`	$329 \times 3 \times 3 \times 3$	stem
`model.1.sequential.0`	$624 \times 329 \times 3 \times 3$	node 0, second conv
`model.1.2.sequential.0`	$329 \times 624 \times 3 \times 3$	node 2, first conv
`model.1.2.1.sequential.0`	$624 \times 329 \times 3 \times 3$	node 2, second conv
`model.1.-1.1.sequential.0`	$329 \times 1248 \times 3 \times 3$	node 1, after concat
`model.1.-1.1.1.sequential.0`	$624 \times 329 \times 3 \times 3$	node 1, second conv

4 Custom Python path in place of Spatha

This section is the implementation that replaces Spatha for the purposes of this project. The code lives in `vnm/vnm_pack.py` and `vnm/prune_eval.py`. There is no new CUDA extension and no change to CoDeepNEAT's `Layer.create_layer`, which still constructs a normal `nn.Conv2d`:

```
deep_layer = nn.Conv2d(channels, self.out_features,
                       window_size, stride, padding)
```

Spatha would have swapped the GEMM *inside* that layer. We swap the **weights**, then leave the layer alone.

4.1 Why a conv must become a matrix

VENOM and Spatha operate on 2-D matrices. A convolution is a 4-D tensor $(C_{\text{out}}, C_{\text{in}}, k_h, k_w)$. Implicit GEMM / im2col already treats it as

$$W_{2D} \in \mathbb{R}^{C_{\text{out}} \times (k_h \cdot k_w \cdot C_{\text{in}})}. \tag{1}$$

C_{in} is placed **innermost** so the column groups of size M run along the same axis as the im2col patches. That is the layout Spatha would multiply against unfolded activations.

```
def conv_to_2d(w: torch.Tensor) -> torch.Tensor:
    c_out, c_in, kh, kw = w.shape
    return w.permute(0, 2, 3, 1).reshape(c_out, kh * kw * c_in)

def conv_from_2d(w2d, c_out, c_in, kh, kw):
    return w2d.reshape(c_out, kh, kw, c_in).permute(0, 3, 1, 2).contiguous()
```

`permute(0, 2, 3, 1)` is "output channel, kernel y, kernel x, input channel." After reshape, row i is output filter i.

Rows are padded to a multiple of V and columns to a multiple of M. The stem has $K = 27$, which does not divide 4, 8 or 16, so that layer's realised sparsity is slightly below the target (48.15% at 2:4 instead of 50%).

4.2 Magnitude V:N:M prune

`prune_vnm_2d` is a one-shot magnitude prune. It is **not** OBS / Fisher (the second-order method in VENOM §6). For each $V \times M$ block:

1. Sum absolute values down the V rows $\rightarrow$ a length-M column score.
2. `topk(..., k=4)` selects the four columns the whole block will keep. Those indices become `col_loc`.
3. Restrict the block to those four columns, then `topk(..., k=2)` **per row**. Those two positions among the four become `m_idx`.
4. Copy the surviving values into a dense matrix of the original shape; everything else is zero.

```
w_blocks = w_pad.view(ns, v, nb, m)
mag = w_blocks.abs().sum(dim=1)            # (n_slabs, n_blocks, M)
_, col_keep = torch.topk(mag, k=4, dim=-1) # 4 columns per block
col_keep, _ = torch.sort(col_keep, dim=-1)
```

```
for s in range(ns):
    for b in range(nb):
        cols = col_keep[s, b]
        slab = w_blocks[s, :, b, :][:, cols]    # (V, 4)
        _, row_keep = torch.topk(slab.abs(), k=2, dim=-1)
        for i in range(v):
            chosen = cols[row_keep[i]]
            pruned[s * v + i, b * m + chosen] = \
                w_pad[s * v + i, b * m + chosen]
```

N is hard-coded to 2 because that is what Sparse Tensor Cores accept. M is 4, 8 or 16 in the sweep. V is 8 (a typical VENOM vector length).

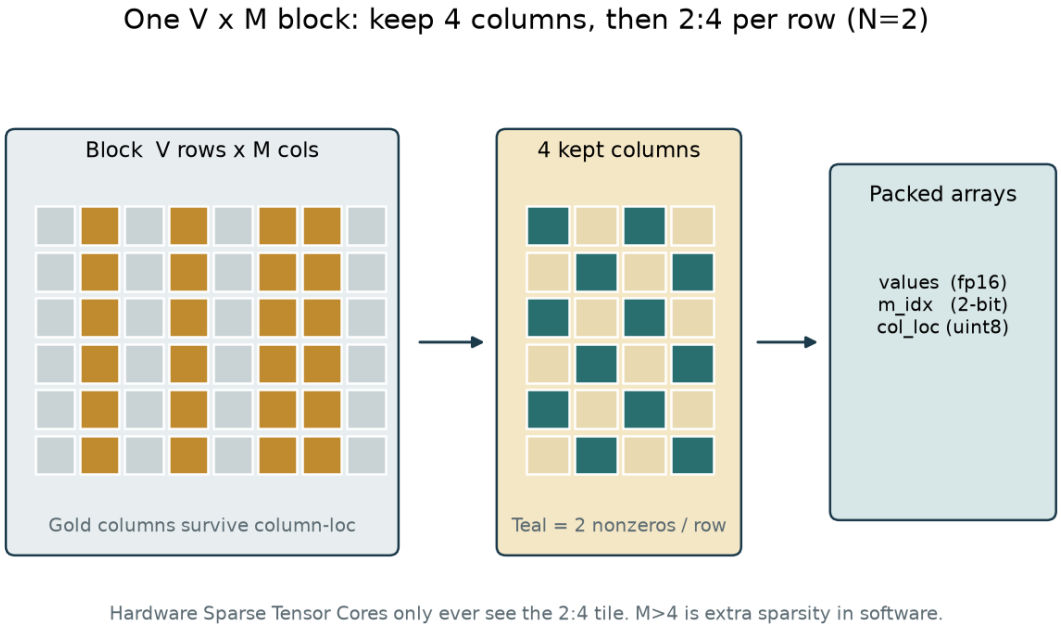


Figure 5: Column-loc drops $M - 4$ columns for the whole slab; then each row keeps two of the remaining four. Packed storage is only those two values plus the two small index arrays.

4.3 Pack and unpack — the format Spatha would consume

`pack_vnm_2d` writes the three arrays a Spatha kernel would read:

```
values  # (R_pad, n_blocks, 2)    surviving weights
m_idx   # (R_pad, n_blocks, 2)    which of the 4 columns, 0..3
col_loc # (n_slabs, n_blocks, 4)  original column ids in 0..M-1
```

`unpack_vnm_2d` inverts that without Python loops. For every row and block it does

```
column = block_index * M + col_loc[row // V, block, m_idx[row, block]]
W[row, column] = values[row, block]
```

We assert `torch.allclose(unpacked, pruned)` so the packer is lossless. `m_idx` and `col_loc` are moved onto `values.device` before indexing; an earlier version left the indices on CPU and crashed when values were on the GPU.

Theoretical size (`packed_nbytes_tight`):

```
bytes = 2 * n_values          # fp16
      + ceil(2 * n_m_idx / 8) # 2-bit indices
      + n_col_loc             # 1 byte, M <= 256
```

The in-memory tensors are wider (`int8` / `int16`) so they are easy to index in PyTorch. The byte count we report is the tight VENOM layout, not the convenient tensors.

4.4 Putting it back into Conv2d — the Spatha substitute

`apply_vnm_to_model` walks `named_modules()`, keeps only `nn.Conv2d`, and for each weight:

```
w2d = conv_to_2d(w)
pruned, mask = prune_vnm_2d(w2d, v=v, m=m, n=2)
packed = pack_vnm_2d(pruned, v=v, m=m, n=2)
rebuilt = unpack_vnm_2d(packed)                  # sanity check
conv.weight.data.copy_(conv_from_2d(pruned, c_out, c_in, kh, kw))
masks[name] = conv_from_2d(mask.float(), ...)
```

After this call the module is still a stock `Conv2d`. Zeros sit in the weight tensor. cuDNN runs a dense convolution. That is the deliberate stand-in (Table 4).

Table 4: Spatha versus the Python substitute used in this project.

	Spatha (not used)	This project
Weight storage	packed `values` / `m_idx` / `col_loc`	same arrays computed, then unpacked
Multiply	CUDA SpMM on Sparse Tensor Cores	`nn.Conv2d` / cuDNN, dense GEMM
Runtime	can be faster on large K	**no speedup** — zeros are still multiplied
What we measure	would be latency	**accuracy + theoretical packed bytes**

We still implement the V:N:M *constraint*. A recover-train step multiplies the weights by the stored mask after every epoch so zeros cannot grow back:

```
def reapply_masks(model, masks):
    for name, mask in masks.items():
        conv.weight.data.mul_(mask.to(conv.weight.device))
```

Fine-tune is Adam, learning rate 10^{-4}, 8 epochs, CIFAR-10 train set.

5 Experimental setup

Colab. Tesla T4, config `colab_baseline`, 12 generations. Winner blueprint **608**, 11.85 M parameters, 60-epoch full-train, $fm = 1$. V:N:M was zero-shot only (no recover-train).

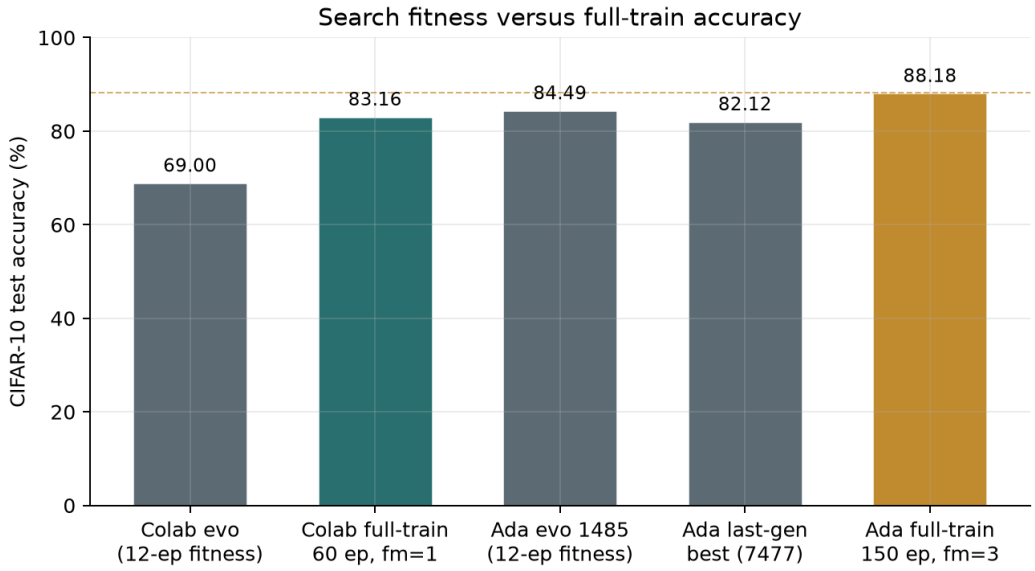
Ada. RTX 6000 Ada (sm 8.9), Python 3.10, PyTorch 2.6 + CUDA 12.4, env `/data2/cs26m305/cdn_env`.

Config `ada_cifar10_strong`. Evolution ran in `tmux` and was resumed after two CUDA OOM events (blueprints hitting the 50 M cap or fragmenting a shared GPU). Full-train: `ft.py`, 150 epochs, three blueprints × two widths. V:N:M: `prune_eval.py --v 8 --ms 4,8,16 --ft-epochs 8` on `bp-1485_fm-3-best-3.model` (11,105,950 parameters).

Data augmentation was **off** in both searches. That caps CIFAR-10 accuracy; industrial numbers (∼95%+) use crop, flip and longer cosine schedules. We did not restart evolution with `evolve_da`.

6 Results

6.1 Search versus full-train



Source: Colab colab_baseline; Ada ada_cifar10_strong. Evo scores are short-train fitness, not 150-epoch accuracy.

Figure 6: Short-train fitness is not the number that goes on a slide. Colab's 69% evo net became 83.16% after 60 epochs. Ada's last generation printed "Best nn 7477, 82.12%"; the network we kept is 1485, which scored 84.49% in its 12-epoch eval at generation 15 and **88.18%** after 150 epochs at $fm = 3$.

1485 was **third** in the evolution ranking. After long training the order flipped (Figure 7, Table 5).

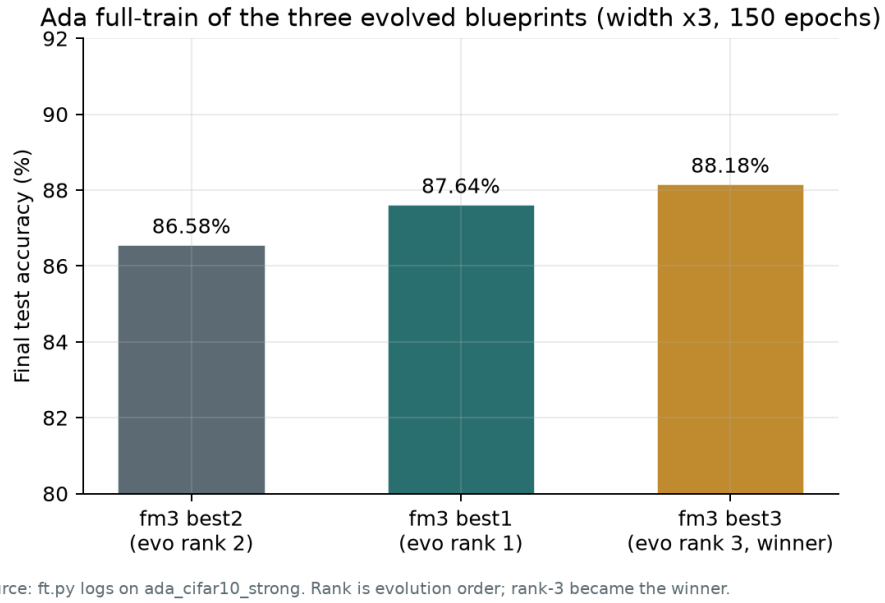


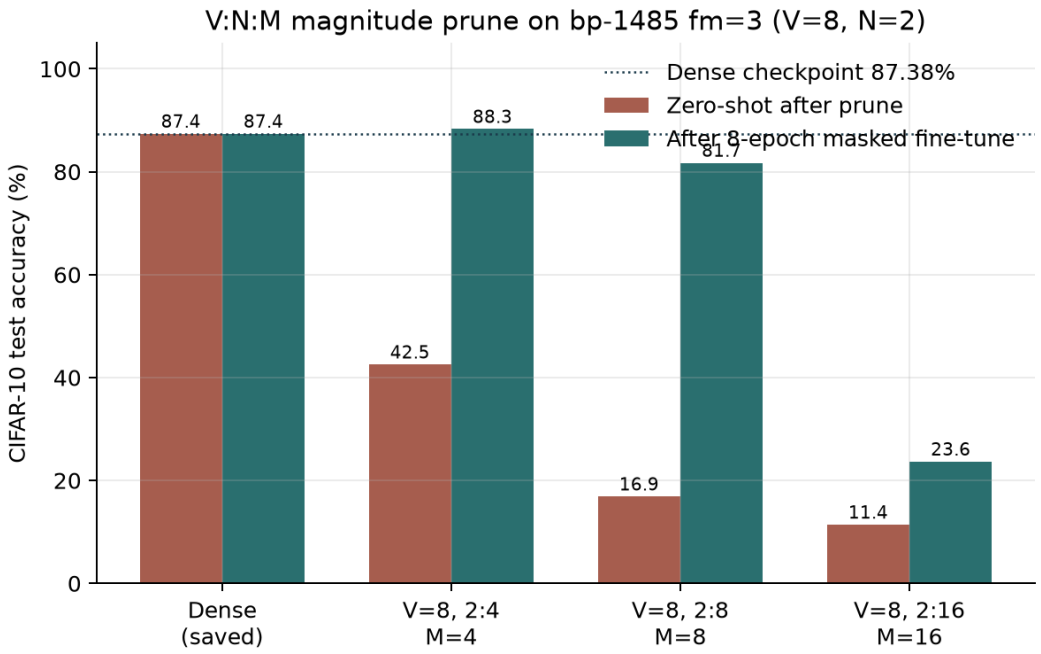
Figure 7: `best-1` / `best-2` / `best-3` are evolution ranks. Rank 3 (`fm3 best3`) is the winner.

Table 5: Ada full-train of the three evolved blueprints at $fm = 3$.

Model	Evolution rank	Test accuracy
fm3 best2	2	86.58%
fm3 best1	1	87.64%
fm3 best3 (bp-1485)	**3**	**88.18%** peak / **87.38%** saved

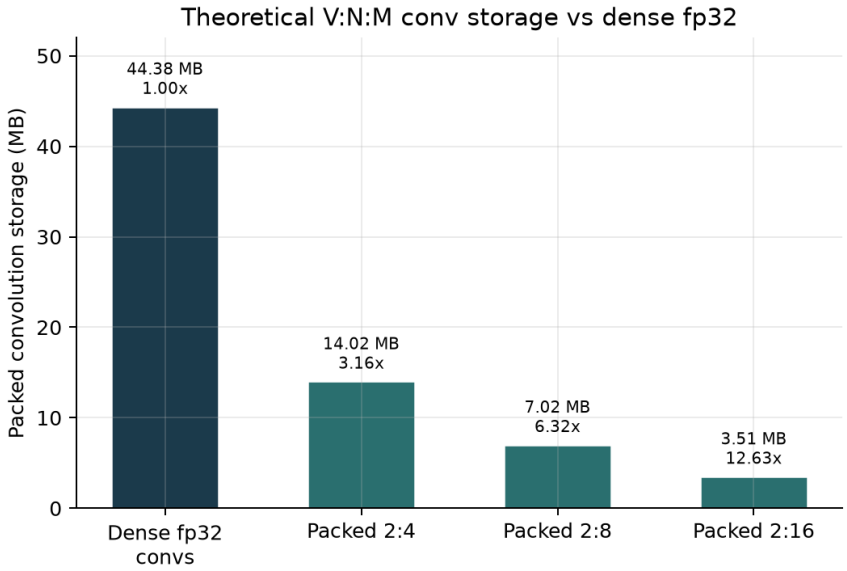
The prune script loads the **saved** weights, so the dense baseline for V:N:M is 87.38%. `ft.py` reports $\max(\text{last}, \text{best-during-train}) = 88.18\%$. Both numbers are real; they are different snapshots.

6.2 V:N:M on the Ada winner



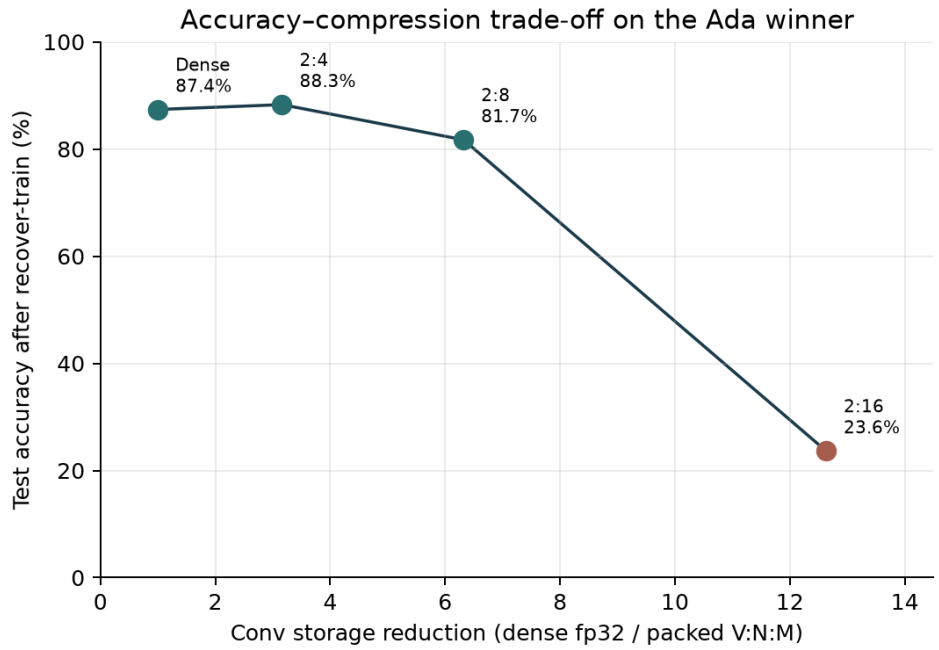
Source: prune_eval.py on bp-1485_fm-3-best-3.model. Dense peak during ft.py was 88.18%.

Figure 8: One-shot magnitude prune collapses accuracy. Eight masked epochs restore 2:4 past the dense checkpoint (88.29% vs 87.38%) and restore 2:8 to 81.72%. 2:16 stays near chance.



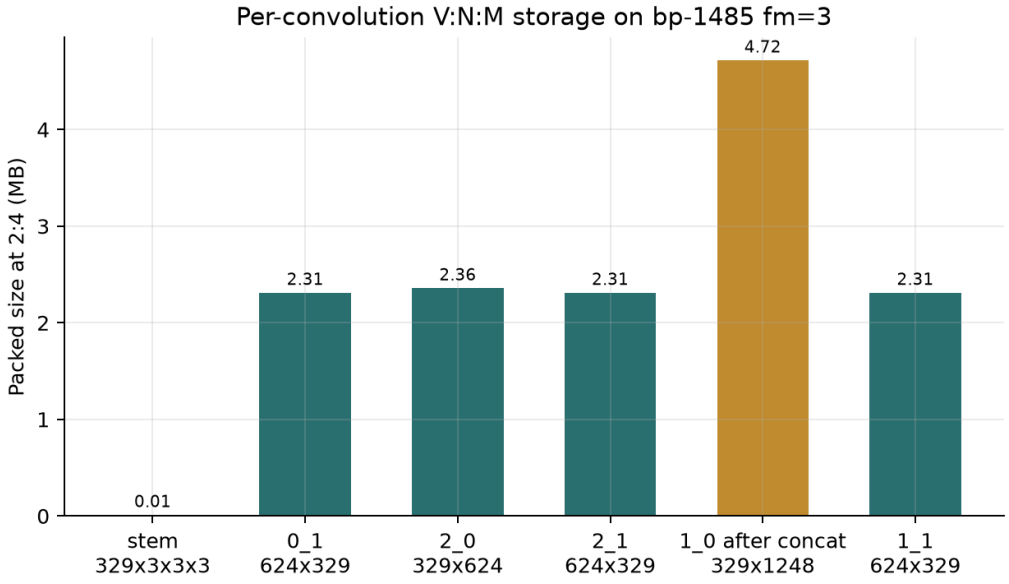
Source: packed_nbytes_tight — fp16 values + 2-bit m_idx + 1-byte col_loc. Biases, BN and the linear head stay dense.

Figure 9: Tight VENOM byte count on the six convolutions only. Dense conv storage is 44,379,468 bytes (fp32).



Source: Ada V=8 sweep. 2:16 (red) does not recover with magnitude prune.

Figure 10: Operating points we would claim: 2:4 (no accuracy loss, 3.16×) and 2:8 (6.3 points down, 6.32×). 2:16 is not a working point under magnitude prune.



Source: prune_eval.py layer dump, V=8 M=4. The 329x1248 layer after concat dominates.

Figure 11: After concat, $329 \times 1248 \times 3 \times 3$ is about a third of packed 2:4 storage. The stem is negligible.

Layer-level 2:4 numbers are in Table 6.

Table 6: Per-convolution packed size at V=8, M=4.

Layer	Shape	Sparsity	Packed bytes
stem	$329 \times 3 \times 3 \times 3$	48.15%	11,760
0_1 / 2_1 / 1_1 (typical)	$624 \times 329 \times 3 \times 3$	49.98%	2,311,920
2_0	$329 \times 624 \times 3 \times 3$	50.00%	2,358,720
1_0 after concat	$329 \times 1248 \times 3 \times 3$	50.00%	4,717,440

6.3 Colab pilot versus Ada

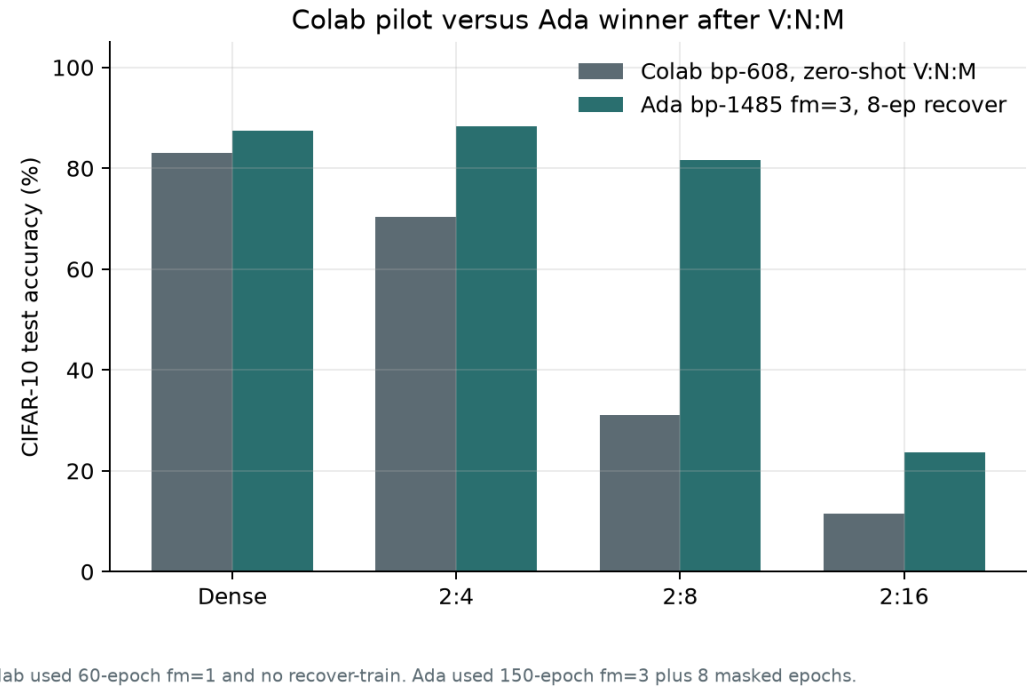


Figure 12: Colab 2:4 was 70.37% **without** recover-train on an 83% dense net. Ada 2:4 is 88.29% **with** recover-train on the 87% dense net. The Ada zero-shot 2:4 (42.51%) is worse than Colab's zero-shot because the wider $fm = 3$ filters are more sensitive to a single magnitude cut; the recover step is what makes the Ada result the headline.

Colab packed sizes (for reference): dense convs 2.26 MB; 2:4 / 2:8 / 2:16 were 3.19× / 6.37× / 12.72× smaller, same format, zero-shot accuracies 70.37 / 31.09 / 11.56%.

6.4 What we are not claiming

- No wall-clock speedup. The GPU still executes dense `Conv2d`.
- No Spatha / Sparse Tensor Core measurement.
- No claim that 2:16 is usable with magnitude prune. VENOM's own high-sparsity results use OBS and a gradual schedule.
- 88% is not a CIFAR-10 SOTA number. It is a no-augmentation CoDeepNEAT CNN after 150 epochs.

7 Discussion

Why 2:4 recovers so well. The mask is a structured 50% pattern the network can adapt to in a few epochs, and BatchNorm re-fits the remaining activations. 2:8 removes three quarters of each block; eight epochs get most of the way back. 2:16 leaves too few weights in small CIFAR filters (K after flatten is 27, 2961, 5616, 11232 — not transformer-sized).

Spatha linking was correctly skipped. CIFAR maps are 32×32 and shrink after pool. SpMM launch cost would likely dominate. The unique result of this project is "an *evolved* CNN can be forced into V:N:M and 2:4 does not lose accuracy," not "we beat cuDNN."

Other prune criteria. Magnitude is what we shipped. OBS / Fisher (VENOM §6) or Taylor scores would be the way to try 2:16 again. That is future work, not a missing bug.

CoDeepNEAT features we used versus unused. Used: module/blueprint coevolution, speciation, multi-eval fitness, feature multipliers, residual/skip via aggregation, Graphviz export, resume-from-pickle. Unused: evolved data augmentation, multi-objective size vs accuracy, DA population.

8 Conclusions

1. CoDeepNEAT on CIFAR-10, with a serious Ada search, produced a skip-concat CNN (blueprint 1485, module 1463) that reaches **88.18%** at width $\times 3$ without evolved augmentation.

2. Convolution weights can be put in VENOM V:N:M by flattening to im2col layout, pruning 4 columns then 2:4 per $V \times M$ block, and packing `values` / `m_idx` / `col_loc`.

3. **Spatha was not linked.** A Python pack/unpack path reconstructs a dense `nn.Conv2d`. That is a substitute for the CUDA kernel for accuracy and storage, not for latency.

4. On the 11.1 M-parameter winner, **2:4 + 8 recover epochs = 88.29% at** $3.16\times$ **smaller conv storage.** 2:8 keeps 81.72% at $6.32\times$. 2:16 fails with magnitude prune.

9 Reproducibility

```
# Evolution (Ada)
cd CoDeepNEAT
python src/main/evo.py -c ada_cifar10_strong -g 1

# Full-train
python src/main/ft.py -c ada_cifar10_strong -g 1

# V:N:M on the winner
python ../vnm/prune_eval.py \
  --run ada_cifar10_strong --config ada_cifar10_strong \
  --v 8 --ms 4,8,16 --ft-epochs 8 \
  --model runs/ada_cifar10_strong/fully_trained_models/bp-1485_fm-3-best-3.
      model
```

Figures in this report are regenerated by `report/generate_figures.py`. Architecture PNGs are copied from `CoDeepNEAT/ada_graphs/top3_g15_blueprint_i1485_{phenotype,genome}.gv.png`.

References

[1] R. Miikkulainen et al., *Evolving Deep Neural Networks*, arXiv:1703.00548.

[2] R. L. Castro et al., *VENOM: A Vectorized N:M Format for Unleashing the Power of Sparse Tensor Cores*, arXiv:2310.02065.

[3] sash-a, *CoDeepNEAT* (PyTorch). https://github.com/sash-a/CoDeepNEAT

[4] UDC-GAC, *VENOM / Spatha*. https://github.com/UDC-GAC/venom

[5] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images* (CIFAR-10).

[6] NVIDIA, *Structured Sparsity on Ampere Sparse Tensor Cores* (2:4).

A Code map

Table 7: Project files used in the CIFAR-10 pipeline.

File	Role
`CoDeepNEAT/src/main/evo.py`	evolution loop
`CoDeepNEAT/src/main/ft.py`	full-train + feature multiplier
`CoDeepNEAT/.../layers/layer.py`	builds `nn.Conv2d` from a module gene
`CoDeepNEAT/.../aggregation_layer.py`	skip / concat
`CoDeepNEAT/.../feature_multiplication.py`	width scaling
`vnm/vnm_pack.py`	flatten, prune, pack, unpack, byte count
`vnm/prune_eval.py`	load winner, apply V:N:M, recover-train, table
`configs/ada_cifar10_strong.json`	80-gen Ada search